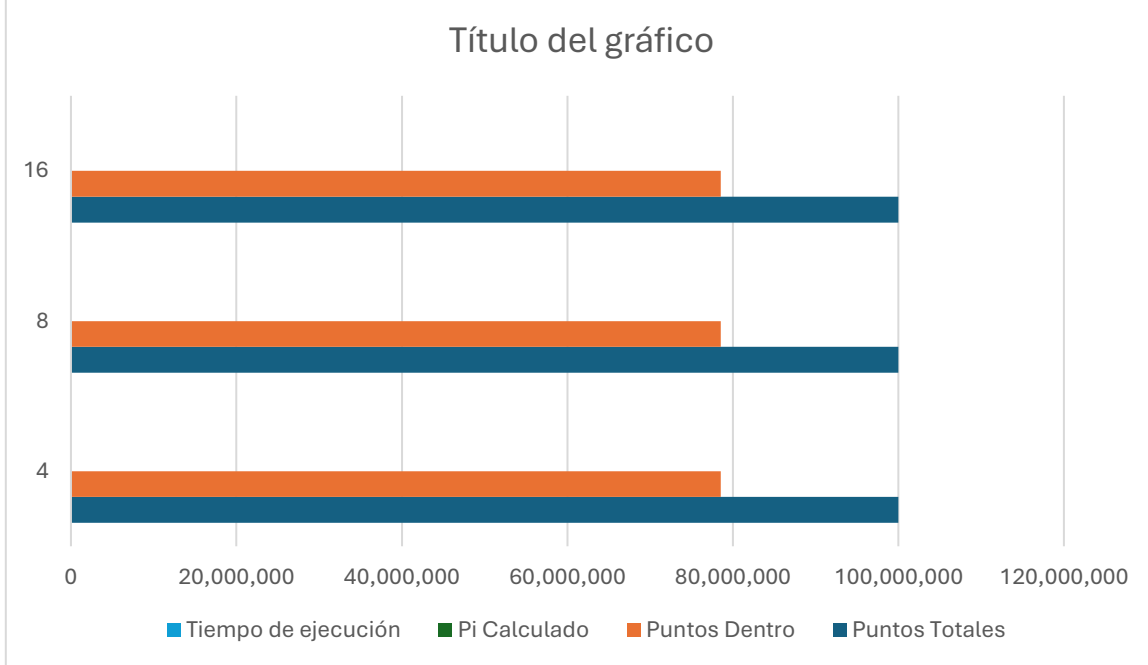


Hilos	Puntos Totales	Puntos Dentro	Pi calculada	Tiempo de ejecución
4	100,000,00	78,535,704	3.141428	0.314486
8	100,000,00	78,539,606	3.141584	0.248064
16	100,000,00	78,534,558	3.141382	0.250102



Explicación

El algoritmo resuelve el problema de exclusión mutua asignando a cada hilo un turno al intentar entrar a la sección crítica, similar a una panadería:

1. **Toma de turno:** Cuando un hilo requiere acceder a la variable compartida puntosDentro, activa su estado `choosing[i] = true`, revisa el turno más alto asignado hasta ese momento y toma el siguiente número disponible ($1 + \max(\text{number})$).
2. **Evaluación de prioridad:** Los hilos ingresan a la sección crítica respetando el orden del número de turno. Si dos hilos obtienen el mismo número al mismo tiempo, el desempate se resuelve favoreciendo al hilo con menor identificador (ID).
3. **Espera activa:** Cada hilo evalúa en un bucle si los demás hilos tienen un turno menor o si están eligiendo número, esperando su turno sin hacer cambios en la variable compartida.
4. **Liberación:** Al terminar de sumar sus puntos acumulados, el hilo restablece su turno a cero (`number[i] = 0`), permitiendo el paso al siguiente hilo en la fila.

Conclusión

Al analizar los tiempos de ejecución observamos que el mejor rendimiento lo obtuvimos con 8 hilos, reduciendo el tiempo a 0.248064 segundos, en comparación con los 0.314486 segundos que tomó la ejecución con 4 hilos

Esta mejora se debe a que acoplar la acumulación de puntos locales por hilo antes de entrar a la sección crítica redujo de forma significativa la competencia por la variable compartida. Sin embargo, al escalar a 16 hilos el tiempo subió ligeramente a 0.250102 segundos. Esto demuestra que, aunque el algoritmo de Lamport permite paralelizar bien las tareas, el incremento excesivo de hilos añade una ligera sobrecarga (overhead) por la gestión de la espera activa y las comparaciones en los arreglos choosing y number.